NB4 — Medallion Pipeline (Bronze → Silver → Gold), lightweight

Use case: LLM observability — exact schema from slide §8 (Lakehouse cho AI/ML) medallion frame. Maps to deliverable bullet 4 (the Milestone-1 Lakehouse artifact).

Pre-req: ran `make data` — but if you jumped straight here, the cell below generates the Bronze sample for you rather than failing on a missing path.

In [1]:
```
import _setup  # noqa: F401  -- adds scripts/ to sys.path
from pathlib import Path

import polars as pl
import duckdb
from deltalake import DeltaTable, write_deltalake
from lakehouse import path, reset

BRONZE = path("bronze", "llm_calls_raw")
SILVER = path("silver", "llm_calls")
GOLD   = path("gold",   "llm_daily_metrics")

# Self-healing pre-req (same pattern as NB7/NB8). Without this, skipping
# `make data` surfaces as a raw `Os { code: 2, kind: NotFound }` from the Rust
# layer — technically correct, useless to a student.
if not Path(BRONZE).exists():
    print("Bronze not found — running scripts/generate_data_lite.py first ...")
    import generate_data_lite

    generate_data_lite.main()
```

Bronze — verify raw is loaded

In [2]:
```
bronze_n = DeltaTable(BRONZE).to_pyarrow_table().num_rows
print(f"Bronze rows: {bronze_n:,}")
print(pl.from_arrow(DeltaTable(BRONZE).to_pyarrow_table().slice(0, 2)))
```

```
Bronze rows: 200,000
shape: (2, 3)
┌──────────────────────────────────────┬──────────────────────────┬──────────────────────────────────────
──────────┐
│ request_id                           ┆ ts                       ┆ raw_json
│
│ ---                                  ┆ ---                      ┆ ---
│
│ str                                  ┆ datetime[μs, UTC]        ┆ str
│
╞══════════════════════════════════════╪══════════════════════════╪══════════════════════════════════════
══════════╡
│ fcdbaa2d-f589-4328-8092-9adc13… ┆ 2026-04-01 00:00:00 UTC ┆ {"model": "claude-sonn
et-4-6",… │
│ 3cef6765-176f-49cf-8318-81481b… ┆ 2026-04-01 00:00:03 UTC ┆ {"model": "claude-haik
u-4-5", … │
└──────────────────────────────────────┴──────────────────────────┴──────────────────────────────────────
──────────┘
```

Silver — parse, validate, dedup

Rules: drop malformed JSON, dedupe by `request_id`, project typed columns.

In [3]:
```python
reset(SILVER)

# DuckDB does the JSON parse + dedup in one query — Polars also works,
# DuckDB just has nicer JSON syntax for this case.
# DuckDB reads Delta through Arrow, not through `delta_scan()`. The latter
# autoloads an extension over the network; Arrow registration is offline and
# zero-copy, so the lab works on a locked-down machine.
con = duckdb.connect()
con.register("bronze", DeltaTable(BRONZE).to_pyarrow_table())

silver_arrow = con.sql(f"""
    WITH parsed AS (
      SELECT
        request_id,
        ts,
        CAST(ts AS DATE)                                AS date,
        json_extract_string(raw_json, '$.model')          AS model,
        json_extract_string(raw_json, '$.user_id')        AS user_id,
        CAST(json_extract(raw_json, '$.usage.input')  AS INTEGER) AS prompt_tokens,
        CAST(json_extract(raw_json, '$.usage.output') AS INTEGER) AS completion_tok
        CAST(json_extract(raw_json, '$.latency_ms')   AS INTEGER) AS latency_ms,
        json_extract_string(raw_json, '$.status')         AS status,
        ROW_NUMBER() OVER (PARTITION BY request_id ORDER BY ts) AS rn
      FROM bronze
    )
    SELECT request_id, ts, date, model, user_id,
           prompt_tokens, completion_tokens, latency_ms, status
    FROM parsed
    WHERE rn = 1 AND model IS NOT NULL
""").arrow()
```

```
write_deltalake(SILVER, silver_arrow, mode="overwrite", partition_by=["date"])

silver_n = DeltaTable(SILVER).to_pyarrow_table().num_rows
print(f"Silver rows: {silver_n:,}  (Bronze {bronze_n:,} → dedup dropped {bronze_n -
assert silver_n < bronze_n, (
    "Silver has the same row count as Bronze — dedup did not run. "
    "Did you regenerate Bronze with the latest generator (which injects retries)?"
)
```

```
Silver rows: 190,052  (Bronze 200,000 → dedup dropped 9,948)
```

Gold — aggregate to (date, model) metrics

In [4]:

```
reset(GOLD)

# Illustrative cost model — NOT canonical pricing.
# (input USD / 1M tokens, output USD / 1M tokens)
COST_TABLE = """
  VALUES
    ('claude-haiku-4-5',  0.80,  4.00),
    ('claude-sonnet-4-6', 3.00, 15.00),
    ('claude-opus-4-7', 15.00, 75.00)
"""

con.register("silver", DeltaTable(SILVER).to_pyarrow_table())
gold_arrow = con.sql(f"""
    WITH cost(model, c_in, c_out) AS ({COST_TABLE})
    SELECT
      s.date,
      s.model,
      QUANTILE_CONT(s.latency_ms, 0.50) AS p50_latency_ms,
      QUANTILE_CONT(s.latency_ms, 0.95) AS p95_latency_ms,
      SUM(s.prompt_tokens)              AS total_prompt_tokens,
      SUM(s.completion_tokens)          AS total_completion_tokens,
      AVG(CASE WHEN s.status <> 'ok' THEN 1.0 ELSE 0.0 END) AS error_rate,
      (SUM(s.prompt_tokens)     * c.c_in  / 1e6) +
      (SUM(s.completion_tokens) * c.c_out / 1e6) AS cost_usd
    FROM silver s
    JOIN cost c USING (model)
    GROUP BY s.date, s.model, c.c_in, c.c_out
    ORDER BY s.date, s.model
""").arrow()

write_deltalake(GOLD, gold_arrow, mode="overwrite", partition_by=["date"])

# Z-order for fast filter-by-model dashboards
DeltaTable(GOLD).optimize.z_order(["model"])
```

Out[4]:
{'numFilesAdded': 7,
 'numFilesRemoved': 7,
 'filesAdded': '{"avg":2769.0,"max":2770,"min":2766,"totalFiles":7,"totalSize":19383}',
 'filesRemoved': '{"avg":2688.1428571428573,"max":2689,"min":2687,"totalFiles":7,"totalSize":18817}',
 'partitionsOptimized': 7,
 'numBatches': 7,
 'totalConsideredFiles': 7,
 'totalFilesSkipped': 0,
 'preserveInsertionOrder': False,
 'plannerStrategy': 'zOrder',
 'preservedStableOrder': False,
 'maxBinSpanFiles': 1}

Verify Gold

In [5]:
```
gold_df = pl.from_arrow(DeltaTable(GOLD).to_pyarrow_table())
print(gold_df)

# Slide-5 deliverable: "Gold p50/p95/cost qua ≥ 7 ngày". Make that explicit.
n_dates = gold_df.select("date").n_unique()
n_models = gold_df.select("model").n_unique()
print(
    f"\n——— Gold deliverable metrics ———\n"
    f"  Distinct dates:   {n_dates:>3}   (target ≥ 7)\n"
    f"  Distinct models:  {n_models:>3}\n"
    f"  Total Gold rows:  {gold_df.height:>3}   (= dates × models)"
)
assert n_dates >= 7, (
    f"Gold has only {n_dates} dates — slide deliverable requires ≥ 7. "
    "Re-run `make data` (the generator spreads across 7 UTC days)."
)
```

shape: (21, 8)

date	model	p50_latency_ms	p95_latency_ms	total_prompt_tokens	total_completion_tokens	error_rate	cost_usd
date	str	f64	f64	decimal[38,0]	decimal[38,0]	f64	f64
2026-04-07	claude-haiku-4-5	566.0	1124.3	16606269	8345341	0.050034	46.666379
2026-04-07	claude-opus-4-7	2993.0	5987.6	5479452	2673162	0.061538	282.67893
2026-04-07	claude-sonnet-4-6	1395.0	2754.85	32420545	16413956	0.048873	343.47097
2026-04-01	claude-haiku-4-5	563.0	1131.0	16722403	8258111	0.048825	46.410366
2026-04-01	claude-opus-4-7	3008.0	6039.4	5436642	2709702	0.050134	284.77728
…	…	…	…	…	…	…	…
2026-04-05	claude-opus-4-7	3098.0	5964.5	5338986	2719749	0.046962	284.06596
2026-04-05	claude-sonnet-4-6	1370.0	2735.0	33110042	16339748	0.052302	344.42634
2026-04-03	claude-haiku-4-5	563.0	1130.45	16298601	8067775	0.048048	45.309981
2026-04-03	claude-opus-4-7	3058.0	5959.9	5529446	2801556	0.045088	293.05839
2026-04-03	claude-sonnet-4-6	1391.0	2744.0	33229930	16596714	0.046037

```
7  ┆ 348.6405  │
│            ┆ net-4-6    ┆           ┆           ┆           ┆           ┆
┆           │
└────────────┴────────────┴───────────┴───────────┴───────────┴───────────┴──────
──┴────────────┘

─── Gold deliverable metrics ───
  Distinct dates:     7   (target ≥ 7)
  Distinct models:    3
  Total Gold rows:   21   (= dates × models)
```

✅ Deliverable check

- ☐ All three tables exist under `_lakehouse/{bronze,silver,gold}/`
- ☐ Silver has fewer rows than Bronze (dedup worked)
- ☐ Gold spans ≥ 7 dates × 3 models (slide §8 medallion contract)
- ☐ Cost & error_rate columns populated and non-zero